

CodeSentinel-LLM:
End-to-End Bug Localization and Patch Generation
with Parameter-Efficient Fine-Tuning

Final-Year B.Tech Major Project Report
8th Sem, 2027

Submitted by

Ashutosh Tripathi
B.Tech Computer Science and Engineering
Indian Institute of Information Technology, Raichur

Under the supervision of

Dr. Neha Agarwal
Department of Computer Science and Engineering
Indian Institute of Information Technology, Raichur

Department of Computer Science and Engineering
Indian Institute of Information Technology, Raichur
2027

Contents

Abstract	3
1 Introduction	4
1.1 Motivation	4
1.2 What the System Does, and What It Does Not Do	4
1.3 Report Structure	4
2 Background and Related Work	5
2.1 Pre-Trained Models for Code	5
2.2 Parameter-Efficient Fine-Tuning	5
2.3 LLM-Based Program Repair	6
2.4 Direct Preference Optimization	6
2.5 Datasets	6
3 Dataset and Preprocessing	7
3.1 Data Sources	7
3.2 Schema	7
3.3 Deduplication	8
3.4 Train / Validation / Test Split	8
3.5 Distribution Analysis	8
4 Architecture	9
5 Bug Localization	10
5.1 Problem Formulation	10
5.2 Architecture	10
5.3 Training	11
5.3.1 Augmentation	11
5.4 Line-Level Evaluation	11
5.5 Results	12
5.6 The Auxiliary Classifier	12
6 Patch Generation: Supervised Fine-Tuning	13
6.1 Localizer-Window Prompting	13
6.2 Model and Training Setup	14
6.3 Training Curves	15
6.4 Evaluation	15
6.4.1 Metrics	15
6.4.2 Results	16
6.4.3 Distribution of Similarity Scores	17
6.5 Qualitative Examples	17
7 Patch Generation: Direct Preference Optimization	18
7.1 Motivation and Setup	18
7.2 Stage 1: Candidate Sampling	18
7.3 Stage 2: DPO Training	19
7.4 Evaluation	19
7.5 Comparison with the SFT Checkpoint	19
7.6 Comparison with a Zero-Shot Baseline	20

8	Observations from Iterative Development	22
8.1	Observation 1: Token-Level F1 Is Not the Right Localization Metric	22
8.2	Observation 2: Augmentation Corrupted Localization Labels	22
8.3	Observation 3: The Classification Task Is Structurally Underspecified	22
8.4	Observation 4: Exact Match Does Not Reflect Repair Quality on This Dataset .	23
8.5	Observation 5: Two-Epoch Overfitting on the SFT Stage	23
8.6	Observation 6: DPO Negatives Without Filtering Are Often Too Strong	23
9	Limitations and Future Work	24
9.1	Standalone Detector	24
9.2	No Execution-Based Validation	24
9.3	The Generator Can Produce Patches That Introduce New Vulnerabilities	24
9.4	Dataset Skew Toward Short Fixes	25
9.5	Inputs Larger Than the Training Distribution	25
9.6	Comparison with Prior Repair Systems	25
10	Conclusion	25
	References	26
A	Hyperparameter Summary	27
B	Prompt Template	27

Abstract

This report presents CodeSentinel-LLM, an end-to-end pipeline for automated detection, localization, and repair of bugs in Python source code. The system is built from two specialized model components that are trained and combined into a single workflow.

The first component is a line-level bug localizer based on GraphCodeBERT, fine-tuned with low-rank adapters (LoRA) on changed-line labels mined from bug-fix commits. Given a code snippet that contains a bug, the localizer produces a ranked list of candidate buggy lines through token-to-line score aggregation. The second component is a patch generator based on Qwen2.5-Coder-7B, fine-tuned in two stages: a supervised stage that learns to produce maintainer-style patches given a windowed prompt containing the buggy region, and a Direct Preference Optimization (DPO) stage that further refines the policy using preference pairs constructed from the model’s own samples.

The dataset is a unified collection of 31,709 Python bug-fix pairs combining TSSB-3M (single-statement bug fixes from open-source commits) and CVEfixes v1.0.8 (security patches indexed by CVE/CWE), split stratified by source and severity into training, validation, and test partitions. The localizer attains a line-level Top-1 accuracy of 0.806, Top-3 of 0.971, and Mean Reciprocal Rank of 0.888 on a held-out test set of 3,101 buggy samples. The supervised generator attains a mean character-level similarity of 0.859 between its outputs and the maintainer-written reference patches, with 73.1% of generations exhibiting high similarity (above 0.8). After DPO refinement, mean character similarity rises to 0.864 and high-similarity rate to 74.5%, with record-level inspection showing improvements outnumber regressions by roughly 1.78 to 1.

The report also documents a number of observations that emerged from the iterative development of the system: misleading aggregate metrics that were diagnosed and replaced, augmentation procedures that silently corrupted labels, an auxiliary classification task that was structurally unsolvable as posed, and concrete failure modes in which the generator produced syntactically plausible but security-degrading edits. A zero-shot baseline using the same model family without fine-tuning attains character similarity 0.593 and exact match 0.066 on the same test set, indicating that the fine-tuning recipe is responsible for the substantial majority of the quality improvement over off-the-shelf behavior. These observations are reported in detail because they shaped the final design and are likely to be encountered by others building similar systems.

Keywords: Automated program repair; bug localization; large language models; LoRA; QLoRA; DPO; Python; vulnerability repair; parameter-efficient fine-tuning.

1 Introduction

1.1 Motivation

Software bugs are an unavoidable part of any non-trivial codebase, and the cost of finding and fixing them dominates much of the engineering effort in real-world software development. Automated program repair (APR) is the long-running research program that aims to reduce this cost by producing software patches automatically given some signal that a bug exists. Classical APR approaches relied on test-suite-guided search over pre-defined edit operators, while modern approaches increasingly use large language models (LLMs) trained on source code to synthesize candidate patches directly from buggy code and natural-language descriptions.

This project investigates a particular slice of the APR problem: *given a Python function or short code window that contains a bug, identify the buggy line and produce a candidate patch.* The chosen slice is narrower than “find and fix any bug in any program,” but it matches the operating mode of standard repair benchmarks such as Defects4J and BugsInPy, where bugs are presumed present and identified at the function or file level by an external triage step. It also matches a common interactive use case in which a developer or static analyzer points at a code region and asks “can this be fixed automatically?”

The system is built end-to-end using publicly available pre-trained models and open datasets. Both model components are fine-tuned using parameter-efficient adapters (LoRA / QLoRA) so that the full training loop is feasible on a single consumer-grade workstation. The combined dataset is constructed from two sources: TSSB-3M, a large mined corpus of single-statement Python bug fixes, and CVEfixes v1.0.8, a curated dataset of security vulnerability patches indexed by CVE and CWE.

1.2 What the System Does, and What It Does Not Do

It is worth being explicit about scope before describing the work in detail. The system takes as input a short Python code window (typically 5–50 lines) and proceeds through three stages. First, a localizer predicts which line(s) within the window are buggy. Second, a window extraction step takes the buggy line(s), pads with surrounding context, and wraps the buggy region with explicit sentinel markers. Third, a patch generator consumes the bracketed window and emits a fixed version of the marked region. The fix is spliced back into the original code to produce the repaired output.

The system does not perform whole-file bug discovery on its own. The training data consists of short bug-fix snippets rather than whole source files, and the localizer is trained to answer “where is the bug in this buggy snippet” rather than “does this code contain a bug.” For deployment on a complete file, an external triage step — for example a static analyzer such as Semgrep or Bandit — is needed to select candidate buggy regions, after which the present pipeline processes each candidate.

The system does not validate patches by executing tests. Validation in this work is similarity-based: a generated patch is compared to the maintainer-written reference patch using several string- and token-level metrics described later. Execution-based validation (running unit tests on the patched code) is an obvious extension and is discussed under future work, but its implementation requires sandboxed Python execution infrastructure that was outside the scope of this project.

1.3 Report Structure

The remainder of this report is organized as follows. Section 2 reviews the relevant prior work in pre-trained code models, LLM-based program repair, and the source datasets used here. Section 3 describes the dataset construction, schema unification, and split protocol. Section 5

details the bug localizer, including the training procedure and the line-level evaluation methodology. Sections 6 and 7 present the supervised and DPO stages of the patch generator respectively. Section 4 describes how the components are combined into the end-to-end repair pipeline. Section 8 consolidates a set of observations made during iterative development that shaped the final system. Section 9 discusses limitations and future work, and Section 10 concludes.

2 Background and Related Work

2.1 Pre-Trained Models for Code

The recent generation of code-understanding and code-generation systems is largely built on top of transformer-based models pre-trained on large corpora of source code.

CodeBERT [1] was an early demonstration that bimodal pre-training over docstrings and code yields representations useful for many downstream tasks including code search, clone detection, and program repair. The model is an encoder of 125M parameters, pre-trained with a masked language modeling objective on the CodeSearchNet corpus.

GraphCodeBERT [2] extends CodeBERT by adding a data-flow-aware pre-training objective. Each input is augmented with its program data-flow graph (which variables are derived from which others), and two additional self-supervised tasks are introduced: edge prediction over the data-flow graph and alignment between code tokens and graph nodes. The result is an encoder whose representations carry richer structural information than CodeBERT alone, with empirically better performance on tasks that depend on local code semantics such as variable misuse detection and defect localization. The present work uses GraphCodeBERT-base (125M parameters) as the backbone of the bug localizer because its data-flow-aware representations are particularly suited to fine-grained, position-sensitive code tasks.

Decoder-only code language models have followed a similar trajectory. **CodeLlama** [3] established 7B–34B parameter open-weight baselines pre-trained on code by continuing the training of Llama 2 on a large code corpus. **DeepSeek-Coder** [4] and **Qwen2.5-Coder** [5] have since extended this direction with stronger pre-training pipelines, infilling capabilities, and broader programming-language coverage. Qwen2.5-Coder-7B in particular is a competitive open-weight choice for Python-centric generation tasks. The present work uses Qwen2.5-Coder-7B-base (i.e., the base pretrained model, not the instruction-tuned variant) as the foundation of the patch generator, fine-tuned via QLoRA so that the trainable parameters are a small fraction of the total.

2.2 Parameter-Efficient Fine-Tuning

Fine-tuning a full 7B-parameter model end-to-end requires updating several gigabytes of weights and the associated optimizer state. **LoRA** [6] (Low-Rank Adaptation) reduces this cost by freezing the base model and learning a low-rank update to selected weight matrices. For a weight $W \in \mathbb{R}^{d \times k}$, LoRA introduces two new matrices $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times k}$ with $r \ll \min(d, k)$, and replaces W at inference by $W + AB$. Only A and B are trained, reducing trainable parameters by orders of magnitude.

QLoRA [7] extends LoRA by additionally quantizing the frozen base weights to 4-bit precision, with the trainable adapters maintained in bfloat16. The compute dtype during forward and backward passes is bfloat16; the 4-bit weights are dequantized on the fly. This combination makes fine-tuning of 7B-class models feasible within the memory budget of a single consumer-grade GPU, and is used throughout the patch-generator training in this work.

2.3 LLM-Based Program Repair

A growing body of work investigates the use of code LLMs for the program-repair task. **Repair-LLaMA** [9] fine-tunes a CodeLlama-7B model with rank-8 LoRA adapters on a curated set of bug-fix pairs, and demonstrates that combining parameter-efficient adapters with repair-specific input representations (in particular, embedding fault localization information directly in the prompt) yields substantial improvements over zero-shot LLM baselines. The benchmarks used are Defects4J, HumanEval-Java, and GitBug-Java, all Java-language datasets evaluated with test-execution metrics.

A parallel line of work investigates LLM-based repair without fine-tuning, using carefully designed prompts and conversational interactions. **AlphaRepair** [12] reformulates patch generation as masked-token infilling, exploiting the pre-training objective directly rather than introducing a separate repair-specific fine-tuning stage. **ConversationalRepair** [11] explores multi-turn dialogue with a frontier-scale LLM, providing test feedback after each candidate patch and producing successively refined attempts. These approaches are complementary to fine-tuning: where fine-tuning specializes a smaller model for a specific task, prompting leverages the general capability of a much larger one.

Reinforcement-learning-based approaches to repair have recently been investigated as well. **SWE-RL** [13] trains policies on repository-scale software-evolution data with execution-based rewards, demonstrating that RL signal from test outcomes can drive meaningful improvements in repair quality. The present work uses DPO rather than execution-based RL, as discussed in Section 7.

2.4 Direct Preference Optimization

Direct Preference Optimization [8] (DPO) is a method for adjusting an already-trained language model so that it prefers one response over another for a given input. The standard formulation begins from a dataset of preference triples (x, y_w, y_l) , where x is a prompt, y_w is the preferred (“winning”) response, and y_l is the dispreferred (“losing”) response. The training objective is:

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \left[\log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right] \right) \quad (1)$$

Here π_{θ} is the policy being trained, π_{ref} is a frozen reference model (typically the SFT checkpoint that the DPO stage starts from), and β controls how strongly the policy is allowed to diverge from the reference. The loss has a natural interpretation: it is the cross-entropy of a binary classifier that must predict which of two responses is preferred, with the logit being the difference in log-probability ratios under policy and reference.

DPO has the practical advantage of not requiring an explicit reward model: the preference data is consumed directly. It is widely used for instruction-tuned LLMs where the preference data comes from human annotators. In code-repair settings, the preference data must typically be synthesized; this is the path taken in Section 7.

2.5 Datasets

TSSB-3M [14] (Single-Statement Bug Fixes) is a large mined corpus of approximately 3 million Python single-statement bug fixes extracted from GitHub commits. Each record consists of a short code window (a few lines of context around the changed line), together with the corresponding pre-fix and post-fix versions. Each fix is automatically classified into one of several syntactic-edit patterns called *sstub patterns* (for example, swapped comparison operators, off-by-one corrections, identifier swaps). The dataset is unusual in scale — few other repair datasets exceed a few thousand records — but its records are short and pattern-specific by construction.

The present work uses a stratified subsample of about 30,000 records drawn from the CTSSB-1M S3-hosted subset of TSSB-3M.

CVEfixes v1.0.8 [15] is a dataset aggregating security-vulnerability patches indexed by CVE (Common Vulnerabilities and Exposures) identifiers and classified by CWE (Common Weakness Enumeration) category. The dataset is distributed as a SQLite database with multiple linked tables; the Python subset is extracted by joining the `method_change` table to itself on (before, after) commits, yielding pre-fix / post-fix function-body pairs together with their CVE and CWE metadata. The present work uses 2,067 such pairs after filtering for valid Python parseability.

The combination of TSSB-3M (large, short, syntactic) and CVEfixes (small, longer, semantically rich) provides a complementary mix: the bulk of the dataset is dominated by short, pattern-shaped fixes that train the localizer-window machinery efficiently, while CVE records contribute meaningful longer-form patches and CWE-category diversity that exercise the model on real security defects.

3 Dataset and Preprocessing

This section describes how the two source datasets were combined into a unified format suitable for training the localizer and the patch generator, and how the resulting records were split into training, validation, and test partitions.

3.1 Data Sources

Two source datasets were initially considered for inclusion: **BugsInPy** [16] and PyBugHive. Both store their bugs as repository snapshots with associated test infrastructure; the actual buggy and fixed code content is accessible only by running `git diff` between specific commit SHAs in checked-out repositories. After investigation, both were dropped: their records do not provide function-level (`buggy_code`, `fixed_code`) pairs as needed for the present pipeline, and the engineering required to reconstruct such pairs (cloning repos, running the test harness per record) was not justified by the modest size of these datasets. The final dataset uses only TSSB-3M and CVEfixes.

TSSB-3M extraction. The CTSSB-1M subset of TSSB-3M is downloaded as a single S3-hosted JSON Lines file. Records are filtered to retain only Python sources (the corpus contains other languages in principle, though Python dominates). A stratified subsample of 30,000 records is then drawn, with strata defined by the `sstbub_pattern` field, so that all pattern categories are represented in approximate proportion to their natural occurrence in the corpus. Stratification avoids the situation in which one or two very common patterns (e.g., variable rename) overwhelm the long tail of less frequent but more interesting patterns.

CVEfixes extraction. The CVEfixes v1.0.8 SQLite database is queried with the following operation: join the `method_change` table to itself on the (`file_id`, `signature`) key, selecting rows where the `before_change` and `after_change` fields together identify a fix commit. The query yields tuples of the form (pre-fix function body, post-fix function body, CVE ID, CWE category, project URL). Records are filtered to require valid Python parseability for both pre-fix and post-fix versions, and a minimum length threshold to exclude empty or trivially short edits. The filter retains 2,067 records out of 2,147 initially extracted.

3.2 Schema

Both sources are mapped into a unified record schema:

```
{
  "id": "ctssb_<commit_hash>_<index>" | "cve_CVE-XXXX-NNNN_<index>"
  ,
```



```

"source":      "ctssb" | "cvefixes",
"buggy_code":  "<Python source string>",
"fixed_code":  "<Python source string>",
"changed_lines": [int, ...],      # 1-indexed lines in buggy_code
"cwe":         "CWE-XX" | null,
"project":     "<owner/repo>" | null,
"code_len_lines": <int>,
}

```

The most significant new field added during unification is `changed_lines`, the list of 1-indexed line numbers within `buggy_code` that differ from the corresponding lines in `fixed_code`. This list is used as the ground-truth label for localizer training: a line is “buggy” if and only if its index appears in `changed_lines`.

For TSSB records the changed line set is provided directly in the source data, having been computed during the original mining process. For CVEfixes the changed line set is computed at preprocessing time using Python’s `difflib.SequenceMatcher`. The matcher is run between the line-split `buggy_code` and `fixed_code` strings, producing a sequence of opcodes (`equal`, `replace`, `delete`, `insert`). For each non-`equal` opcode, the lines of `buggy_code` that fall within its span are added to the changed set. The union across all such opcodes is the final `changed_lines` value for the record.

3.3 Deduplication

Records can be duplicated across the two sources for several reasons: the same commit may be referenced by both TSSB and CVEfixes (uncommon but possible), or near-identical small fixes may appear multiple times across repositories using shared code. A content-based deduplication pass is applied: each record’s (`buggy_code`, `fixed_code`) pair is hashed with SHA-256, and records with collisions are dropped. This reduces the raw union of 31,973 records to a final 31,709 deduplicated records.

3.4 Train / Validation / Test Split

The deduplicated records are partitioned into training, validation, and test sets with target proportions 80% / 10% / 10%. The split is stratified jointly by source (`ctssb` or `cvefixes`) and by changed-line count bucket, ensuring that both sources are represented in each partition with their natural proportions and that no partition is dominated by an unusual edit-length distribution.

The final partition sizes are: **train 25,451 records, validation 3,129 records, test 3,129 records**. Table 1 reports the per-source / per-split composition. TSSB-derived records dominate by volume (94.3%), reflecting the much larger source corpus.

Table 1: Dataset composition by source and split.

Source	Train	Val	Test	Total
CTSSB	23,920	2,993	2,993	29,906
CVEfixes	1,531	268	268	2,067
Total	25,451	3,129	3,129	31,709

3.5 Distribution Analysis

Table 2 reports the distribution of `buggy_code` line counts across the full dataset. The most striking feature is the strong concentration of records in the 5–9 line range (89.2% of the dataset).

This reflects TSSB’s design: each TSSB record is a short diff-context window extracted from the original commit, rather than the full enclosing function. CVEfixes records, which contain full function bodies, are responsible for most of the records in the longer buckets.

Table 2: Line-count distribution of `buggy_code` across all 31,709 records.

Line count	Records	% of dataset
0–4	1,691	5.3%
5–9	28,273	89.2%
10–19	654	2.1%
20–39	575	1.8%
40–79	313	1.0%
80–159	148	0.5%
160+	55	0.2%

This distribution shape constrains downstream design choices: prompt and target sequences in training are short on average, and the system can be tuned to work efficiently with relatively small context windows.

For the CVEfixes subset, Table 3 reports the top CWE categories represented in the data. Coverage of common web-application vulnerability classes is broad, with path traversal (CWE-22), server-side request forgery (CWE-918), and cross-site scripting (CWE-79) particularly well-represented. The diversity of CWE categories is important because each category corresponds to a different family of fix patterns: SQL-injection fixes typically introduce parameterized queries, path-traversal fixes typically introduce input-sanitization functions, deserialization fixes typically replace untrusted-input pickle calls with safer parsers, and so on.

Table 3: Top CWE categories in the CVEfixes Python subset.

CWE	Description	Records
CWE-918	Server-Side Request Forgery (SSRF)	148
CWE-79	Cross-Site Scripting (XSS)	136
CWE-22	Path Traversal	94
CWE-94	Code Injection	53
CWE-352	Cross-Site Request Forgery (CSRF)	41
CWE-89	SQL Injection	40
CWE-502	Deserialization of Untrusted Data	37

4 Architecture

Figure 1 shows the pipeline architecture as a block diagram. The detector component is shown as a dashed box because it is not part of the trained system in the present work; instead, an external static analyzer such as Semgrep or Bandit fills this role at deployment time, or the user manually selects a region.

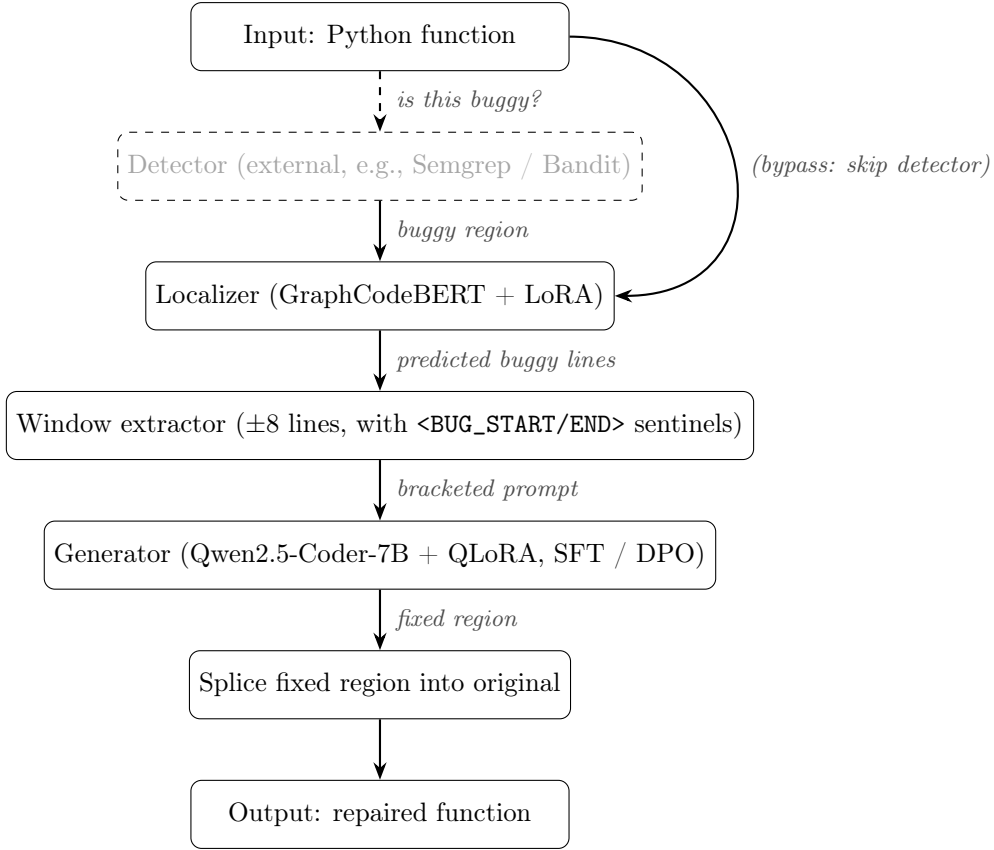


Figure 1: End-to-end pipeline architecture. The detector component (dashed) is provided externally at deployment time, e.g., by a static analyzer such as Semgrep or Bandit, or bypassed entirely when the user supplies a code region known or suspected to contain a bug.

5 Bug Localization

This section describes the bug localizer, which is the first component of the end-to-end pipeline. Given a code snippet that contains a bug, the localizer’s task is to identify which line(s) within the snippet are responsible for the bug.

5.1 Problem Formulation

The localizer is formulated as a per-token binary classification problem followed by a token-to-line aggregation step. Each input token is labeled as either inside the buggy region (positive class) or outside it (negative class), where the buggy region is defined as the lines listed in `changed_lines`. The model produces a per-token probability distribution; these probabilities are then aggregated up to lines (Section 5.4) to produce a ranked list of candidate buggy lines.

Formulating localization as a token-level task rather than a direct line-level task has two practical advantages. First, the transformer backbone operates naturally on tokens; computing a per-line representation would require either pooling or a separate line-segmentation step. Second, token-level supervision gives the model gradient signal at every position, rather than only at line boundaries, which empirically yields more stable training.

5.2 Architecture

The backbone is **GraphCodeBERT-base** [2]: a 12-layer transformer encoder with hidden size 768, 12 attention heads per layer, and 125M total parameters. LoRA adapters are inserted into

the **query** and **value** projections of every attention layer, with rank 16, $\alpha = 32$, and dropout 0.05. The trainable parameter count is 0.42M out of 125.5M total (approximately 0.33%).

Two task heads sit on top of the backbone:

- A *localization head*: a 2-way linear classifier applied per-token (inside-bug vs. outside-bug). The loss is class-weighted cross-entropy with the positive class weighted by a factor of 10, to counter the substantial class imbalance — a typical buggy snippet has one or two buggy lines out of five to nine total, so most tokens are class-0.
- A *classification head*: a 2-way linear classifier applied to the pooled CLS token (snippet-is-buggy vs. snippet-is-clean). This head was included for joint training but did not produce a usable signal at evaluation time, for reasons discussed in Section 5.6.

5.3 Training

Training uses a composite loss that sums the per-token localization loss and the snippet-level classification loss. CVEfixes-derived samples are upweighted by a factor of 5 during loss accumulation to counter their underrepresentation relative to TSSB. The optimizer is AdamW with weight decay 0.0 and a cosine learning-rate schedule with 3% warmup; the base learning rate is 2×10^{-4} . The model is trained for 6 epochs in mixed-precision (bfloat16) computation, with composite early-stopping based on a weighted combination of validation localization F1 and validation classification F1.

5.3.1 Augmentation

Standard augmentations are applied to the training data to improve backbone-level generalization: random whitespace perturbations (A1), consistent identifier renaming applied to both buggy and fixed sides (A2), insertion of blank lines (A4), and insertion of comments (A7). These augmentations expand the effective training-set diversity without changing the semantics of the code.

A non-trivial implementation detail is that augmentations A1, A2, A4, and A7 modify the `buggy_code` text but do *not* update the `changed_lines` label list. A blank-line insertion above line k shifts line k to line $k + 1$ in the text, but the label still points at the old position. This was an early source of label corruption that was identified and fixed by restricting the localization-loss computation to non-augmented samples only. The classification loss continues to see augmented samples (since it does not depend on line positions), and the backbone benefits from both. This issue is discussed in more depth in Section 8.

5.4 Line-Level Evaluation

The localizer is evaluated at the level of lines, not tokens. The reason is that token-level metrics (e.g., per-token accuracy or F1) are misleading on this dataset: in a typical 7-line buggy snippet with one buggy line, roughly 85–90% of tokens belong to the majority (not-buggy) class, and even a degenerate classifier that predicts “not buggy” for every token attains very high token-level F1. The metric does not capture what is actually wanted from the localizer — namely, that the buggy line is surfaced as a top candidate.

The line-level evaluation procedure is the following.

1. Forward-pass the snippet through the trained model to obtain the per-token buggy-class probability $p_i \in [0, 1]$ for each token position i .
2. Use the tokenizer’s `offset_mapping` (which maps each token back to its character span in the source string) to assign each token to a line of the original snippet.

3. Compute a per-line score as the mean of the probabilities of all tokens belonging to that line. (Mean rather than max because mean is empirically more robust to occasional spurious high-probability tokens.)
4. Sort the lines by score in descending order to produce a ranked list $L_1, L_2, \dots$
5. For a sample whose true buggy lines are the set G :
 - **Top- K accuracy** is 1 if any of $L_1, \dots, L_K$ falls in G , else 0.
 - **Reciprocal rank** is $\frac{1}{\min_i \{i: L_i \in G\}}$, with the convention that the reciprocal rank is 0 if no L_i is in G .
6. Aggregate by averaging over all samples in the evaluation set to obtain Top- K accuracy and Mean Reciprocal Rank (MRR).

These line-level metrics correctly reward the system for ranking the buggy line among its top predictions, regardless of how the remaining benign tokens are scored.

5.5 Results

The localizer is evaluated on the held-out test set, restricting to the 3,101 buggy samples for which at least one ground-truth buggy line is visible in the snippet after tokenization (a small number of records have their changed lines pushed outside the visible window by tokenizer truncation, and are excluded).

Table 4 reports the headline metrics. The Top-3 accuracy of 0.971 is the most operationally important number: when the downstream patch generator considers more than one candidate window (as in the production pipeline), the relevant question is whether the gold buggy line is among the top few candidates rather than strictly the top one. The Mean Reciprocal Rank of 0.888 is consistent with this picture — the average sample has the buggy line near the top of the ranked list.

Table 4: Line-level localization performance on the held-out buggy test set ($n = 3,101$).

Metric	Value
Top-1 accuracy	0.806
Top-3 accuracy	0.971
Top-5 accuracy	0.986
Mean Reciprocal Rank	0.888

5.6 The Auxiliary Classifier

The auxiliary snippet-level classifier head, which was intended to answer “does this snippet contain a bug,” does not produce a usable signal: it predicts “buggy” for virtually every input it sees, with F1 of approximately 0.665 and accuracy of 0.500 on a balanced held-out set.

This is not a training failure in any technical sense — the loss decreases monotonically during training and the model fits its training data well. The problem is that the classification task as posed by the dataset is structurally underspecified. The classifier sees pairs (`buggy_code`, `fixed_code`) that differ by only one or two tokens out of fifty to two hundred, and the same code phrasing (for example `if x < 10:`) appears as a positive example in one record and a negative example in many other records. There is no within-snippet signal that distinguishes which side of the fix the model is being shown; the answer depends entirely on context that is not provided.

For the present pipeline, the classifier head’s output is disregarded. The localizer operates in conditional mode: it answers “where is the bug, assuming one exists” rather than “does this code contain a bug.” Producing a properly calibrated detector would require training on different data — specifically, on negatives that are not microedits of positives but rather genuinely distinct functions known to be bug-free. This is discussed under future work in Section 9.1.

6 Patch Generation: Supervised Fine-Tuning

This section describes the supervised fine-tuning stage of the patch generator (referred to as Phase 4a in the project). The goal is to produce, from a buggy code window, a candidate fix region that replaces the buggy line(s).

6.1 Localizer-Window Prompting

A naive way to train a patch generator is to feed the entire `buggy_code` as input and ask the model to produce the entire `fixed_code` as output. This approach has several drawbacks: the input contains a great deal of unchanged context that contributes little to the fix; the output is long and the alignment between input and output is implicit; and inference cost scales with the full snippet length.

The approach used here, *localizer-window prompting*, instead constructs a small window around the buggy region and marks the buggy lines with explicit sentinels. Given a record with `buggy_code`, `fixed_code`, and `changed_lines`, the procedure is:

1. Extract a window from `buggy_code` consisting of the buggy lines together with ± 8 lines of surrounding context, capped at 80 total lines and padded out to at least 6 lines if the snippet is unusually short.
2. Insert the sentinel string `# <BUG_START>` immediately before the first buggy line and `# <BUG_END>` immediately after the last buggy line. These are valid Python comments, so they do not interfere with code parseability if the model attempts any intermediate AST inspection.
3. Compute the corresponding fix region in `fixed_code` by running `difflib.SequenceMatcher` between the line-split `buggy_code` and `fixed_code`, and selecting the fixed-side line range that aligns with the buggy region.
4. Construct the training prompt and target as illustrated in Figure 2: the prompt instructs the model to emit only the corrected version of the marked region, and the target is exactly that fixed region.

Design rationale. The sentinels serve three purposes. First, they communicate the localizer’s prediction to the generator using ordinary code-comment syntax that the model has seen many times during pretraining, rather than introducing special tokens that would require vocabulary changes. Second, they make the generation target unambiguous: the fix is the analog of the bracketed region, no more and no less. Third, they make the splice step at inference trivial: the generator’s output substitutes directly for the bracketed lines in the original code, with no string alignment or diff machinery needed.

Loss masking. During training, the cross-entropy loss is computed only on the target tokens (the fixed region and the closing code fence). The prompt tokens are present in the input for the attention computation but are masked out of the loss with the value -100 in the labels tensor.

Prompt:

```

### Task:
Fix the bug in the following Python code. The buggy region is marked with
# <BUG_START> and # <BUG_END>. Output ONLY the corrected version.

### Buggy code:
(eight lines of context above)
# <BUG_START>
    src_uri = 'file://' + os.path.join(DATAROOT, dataset, rel_path)
# <BUG_END>
(eight lines of context below)

### Fixed region:

```

Target:

```
src_uri = 'file://' + _get_obj_absolute_path(dataset, rel_path)
```

Figure 2: Localizer-window prompting: the generator sees a buggy code window in which the buggy region is bracketed by sentinels, and is trained to emit only the corrected version of the bracketed region. The example is from CVE-2022-31506.

6.2 Model and Training Setup

The base model is Qwen2.5-Coder-7B-base [5], the base (not instruction-tuned) variant. The model is loaded in 4-bit precision via the bitsandbytes library using NF4 double-quantization with bfloat16 as the compute dtype. LoRA adapters are placed on all linear projections in every transformer block: `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj`. The LoRA rank is 16, $\alpha = 32$, and dropout is 0.05. The resulting trainable-parameter count is 40.4M out of 4.39B total weights (approximately 0.92%).

Training hyperparameters are listed in Table 5.

Table 5: SFT hyperparameters.

Parameter	Value
Base model	Qwen/Qwen2.5-Coder-7B
Adapter type	QLoRA (4-bit NF4, bf16 compute)
LoRA rank	16
LoRA alpha	32
LoRA dropout	0.05
LoRA target modules	q,k,v,o,gate,up,down_proj
Trainable parameters	40.4M (0.92% of total)
Epochs	2
Batch size	4
Gradient accumulation	4 (effective batch 16)
Learning rate	2×10^{-4}
Schedule	Cosine, 3% warmup
Weight decay	0.0
Optimizer	AdamW
Max sequence length	1024 tokens
Context lines per side	8
Training time	approximately 2 hours

6.3 Training Curves

Table 6 reports the per-epoch training and validation losses. The training loss decreases substantially between the two epochs ($0.381 \rightarrow 0.211$), while the validation loss *increases* ($0.367 \rightarrow 0.402$). This is a clear signature of overfitting: the model is learning to fit the training records more tightly at epoch 2, but the validation behavior is degrading.

Table 6: SFT training curves. Validation loss rose in epoch 2, indicating overfitting; the best checkpoint is therefore taken from epoch 1.

Epoch	Train loss	Val loss
1	0.381	0.367
2	0.211	0.402

The system tracks best-validation-loss across epochs and saves the corresponding checkpoint separately under a `best/` subdirectory; this checkpoint comes from epoch 1 and is the checkpoint used in all subsequent evaluation.

The fact that overfitting sets in so quickly is informative. The Qwen2.5-Coder-7B-base model already speaks fluent Python from its pretraining; the role of supervised fine-tuning here is primarily to teach the localizer-window output format rather than to teach Python syntax or general fix patterns. The format is simple enough to be absorbed in a single epoch, and continued training beyond that point predominantly causes the LoRA adapters to memorize training-record idioms at the expense of generalization. This pattern is consistent with reports for other LoRA-tuned code-repair systems and is the reason the training schedule was intentionally kept short.

6.4 Evaluation

The patch generator is evaluated on the held-out test set of 3,129 records. For each record, the window-extraction procedure described above is applied to construct the prompt; a small number of records fail extraction (e.g., because the labeled changed line falls outside the visible window after tokenizer truncation) and are excluded from evaluation. After this filtering 3,112 records are evaluated.

For each evaluated record, the model is prompted and produces a candidate fix region via greedy decoding (`do_sample = False`, beam width 1, maximum 256 new tokens). The generated text is post-processed to strip the closing code fence, and the cleaned output is compared to the gold fix region with several metrics.

6.4.1 Metrics

Several complementary metrics are reported for each generation:

- **Exact match (EM).** The generation is correct if and only if it is byte-identical to the gold fix region after stripping leading and trailing whitespace.
- **Normalized exact match.** EM after additionally collapsing all whitespace runs to single spaces. This is more lenient about formatting differences than raw EM.
- **Character-level similarity.** Computed using `difflib.SequenceMatcher.ratio()`, this is a value in $[0, 1]$ measuring the fraction of matching characters in the longest common subsequence between gold and generated strings, suitably normalized. Character-level similarity is a standard similarity-based metric that captures partial credit for generations that are close to but not identical to gold.

- **Token-overlap F1.** Both gold and generated strings are tokenized into Python code tokens (identifiers, keywords, operators, punctuation) and treated as multisets. The metric is the F_1 score between these multisets: precision is the fraction of generated tokens that appear in gold (with multiplicity), recall is the fraction of gold tokens that appear in the generation, and F_1 is their harmonic mean. This measures lexical overlap independent of token ordering.
- **High-similarity rate at threshold τ .** The fraction of generations whose character similarity equals or exceeds τ . We report at $\tau = 0.5$ (close to gold) and $\tau = 0.8$ (very close to gold).
- **Truncation rate.** A heuristic estimate of the fraction of generations that were cut off mid-statement by the `max_new_tokens` budget. The heuristic checks whether the generation ends at a sensible terminator character (closing parenthesis, bracket, colon, etc.) versus mid-identifier.

Reporting multiple metrics rather than a single headline number is important because the metrics tell complementary stories: EM is strict and unforgiving, character similarity gives partial credit, token-overlap F1 is order-independent, and the high-similarity rate quantifies the long tail of close-but-not-exact generations.

6.4.2 Results

Table 7 reports the metrics aggregated over the full test set and broken down by source.

Table 7: Supervised generator evaluation on the held-out test set ($n = 3,112$). Pass@1 with greedy decoding.

Metric	Aggregate	CTSSB	CVEfixes
n	3,112	2,938	174
Exact match	0.225	0.226	0.213
Normalized EM	0.226	0.226	0.218
Character similarity (mean)	0.859	0.865	0.756
median	0.898	0.900	0.817
75th percentile	0.987	0.987	0.976
High-similarity ≥ 0.8	0.731	0.743	0.523
High-similarity ≥ 0.5	0.970	0.978	0.851
Token-overlap F1 (mean)	0.817	0.820	0.772
Truncation rate (heuristic)	0.111	0.107	0.184

The headline observations are the following.

Strong aggregate similarity, modest exact match. The mean character similarity of 0.859 and the high-similarity rate of 0.731 indicate that the model’s outputs are generally very close to the maintainer’s fixes. The exact-match rate of 0.225 is lower in absolute terms because EM penalizes any deviation from the byte-exact gold string, including minor formatting differences and the use of semantically equivalent alternative constructions. The two numbers together describe the model’s behavior more completely than either alone.

Source-specific variation. CTSSB single-statement fixes are substantially easier than CVEfixes patches. Mean character similarity on CTSSB is 0.865 with 74.3% high-similarity; on CVEfixes it is 0.756 with 52.3% high-similarity. CTSSB records typically involve a single small lexical change (e.g., swapping an operator, renaming a parameter), and the model rarely has trouble reproducing them. CVE patches frequently span multiple lines and involve domain knowledge about specific weakness classes; producing a maintainer-matching fix is harder.

Truncation matters on CVEfixes. The heuristic truncation rate of 18.4% on CVEfixes (versus 10.7% on CTSSB) suggests that the 256-token output budget is occasionally binding for

longer multi-line security patches. Increasing the budget would likely recover some of this gap, at modest additional inference cost.

6.4.3 Distribution of Similarity Scores

Beyond aggregate means, the full distribution of character-similarity scores tells a more complete story. Table 8 bins the 3,112 generations by character similarity.

Table 8: Distribution of character-similarity scores across all 3,112 test generations.

Similarity bucket	Count	Percentage
0.00 – 0.20 (very low)	5	0.2%
0.20 – 0.50 (low)	87	2.8%
0.50 – 0.70 (medium)	356	11.4%
0.70 – 0.85 (high)	728	23.4%
0.85 – 0.99 (very high)	1,186	38.1%
0.99 – 1.00 (near-exact / exact)	750	24.1%

Approximately 62% of generations fall in the “very high” or “near-exact” buckets (similarity ≥ 0.85), and only 3% fall in the “very low” or “low” buckets (similarity < 0.5). The distribution is heavily concentrated toward the high-similarity end, consistent with the qualitative impression that the model is doing something useful on most inputs.

6.5 Qualitative Examples

Aggregate metrics necessarily compress information. To convey the range of behaviors the system exhibits in practice, three representative samples are described in detail below.

Example 1: Alternative valid security fix (CVE-2024-31461). The vulnerability is a hostname-validation bypass: a downstream function accepts an arbitrary hostname and the original code did not restrict it to an allowlist of trusted domains. The maintainer’s fix uses `urlparse` to extract the base domain from the input, then checks that the base domain is in an allowed list:

```
allowed_domains = ["atl-paas.net", "atlassian.com",
                  "atlassian.net", "jira.com"]
parsed_uri = urlparse(f"https://{hostname}")
domain = parsed_uri.netloc.split(":")[0]
base_domain = ".".join(domain.split(".")[2:])
return base_domain in allowed_domains
```

The model’s generation produces essentially the same logic with slightly different comments and arrangement, also using `urlparse` and the same allowlist semantics. The character similarity is 0.753 and EM is False, but the two implementations are functionally equivalent and both correctly defend against the vulnerability.

Example 2: Near-miss with defensive variation (single-statement fix). The original line is `if isinstance(node[-1], nodes.Element):`, and the maintainer left this exact form. The model generated `if len(node) > 1 and isinstance(node[-1], nodes.Element):`, adding a defensive length check before the negative-index access. The two are not identical (EM = False, character similarity 0.85), but the model’s version is arguably more robust: it avoids a possible `IndexError` if `node` is empty. From a code-review standpoint this would likely be accepted.

Example 3: A concerning failure (CVE-2022-23071). The maintainer’s fix for a server-side request forgery vulnerability adds explicit URL validation before fetching a remote image:

```
url = file['originalPicture']
if validators.url(url, public=True):
    response = requests.get(url)
...
```

The model’s generation, instead of validating the URL, sets `verify=False` on the request:

```
response = requests.get(file['originalPicture'], verify=False)
...
```

Disabling TLS verification is not a valid alternative to URL allowlisting — it is, in fact, a separate security weakness. The generation is syntactically plausible and superficially appears to modify the relevant code, but it does not address the original vulnerability and introduces a new one of comparable severity. The implication for any deployment is that generated patches require human review, particularly for security-sensitive code. This is discussed further in Section 9.

7 Patch Generation: Direct Preference Optimization

This section describes the Direct Preference Optimization (DPO) stage of the patch generator (referred to as Phase 4b in the project). The supervised checkpoint from Section 6 is the starting point; the DPO stage produces a refined policy trained to prefer maintainer-style fixes over alternatives sampled from its own distribution.

7.1 Motivation and Setup

The DPO loss requires a dataset of preference pairs (x, y_w, y_l) , where x is a prompt, y_w is the preferred (winning) response, and y_l is the dispreferred (losing) response. For chat instruction tuning, preference pairs are typically obtained from human annotators who rank candidate responses. No such annotators were available for the present work, so preference pairs were constructed synthetically from the model’s own samples.

The construction proceeds in two stages, described below.

7.2 Stage 1: Candidate Sampling

For each prompt in the training set, four candidate completions are generated from the SFT model using temperature sampling with $T = 1.2$, $\text{top-}p = 0.95$, and $\text{top-}k = 80$. The high temperature is deliberate: at low temperature, the SFT model produces nearly deterministic outputs that are essentially equivalent to the greedy prediction, and four such samples would be too similar to provide useful contrast as preference negatives. High-temperature sampling introduces diversity into the candidate pool at the cost of occasionally producing degenerate completions.

Each candidate is scored by computing its character-level similarity to the gold fix region. The intuition is that a candidate with low character similarity to gold is, on average, a worse fix than one with high similarity — not perfectly correlated with semantic correctness, but a reasonable proxy in the absence of test execution.

Filtering. For each prompt, the worst of the four candidates (the one with lowest character similarity to gold) is retained as a candidate for use as y_l . A subsequent filtering pass discards any prompt for which even the worst candidate has character similarity to gold ≥ 0.70 . This filter is important: when even the worst candidate is reasonably close to gold, treating it as a “losing” example in DPO would teach the model to reject valid alternative fixes (such as Example 1 in Section 6.5, where the model produced a functionally equivalent but lexically different patch). This kind of training signal would degrade the model’s overall quality.

The filter retains 10,371 preference pairs out of 25,330 prompts (40.9% retention). The distribution by source is 9,130 from CTSSB and 1,241 from CVEfixes; CVEfixes retains a higher fraction of prompts because its longer, more complex patches admit more diverse high-temperature samples. The mean character similarity of the retained y_l candidates is 0.529, well below the 0.70 threshold, indicating that the filter is selecting genuinely bad candidates rather than borderline cases.

7.3 Stage 2: DPO Training

For each retained preference pair, y_w is the gold fix region and y_l is the worst candidate from Stage 1. Both y_w and y_l are formatted using the same prompt template as during SFT.

The DPO loss is implemented directly (without using a library wrapper, for transparency about what is happening inside the training loop). The implementation centers on the per-response log-probability sum: given input ids and labels (where prompt positions are masked with -100), the model is forward-passed and the log-softmax over the vocabulary is computed at every position; the log-probability of the correct next token at each non-masked position is summed to produce $\log \pi(y | x)$. The DPO loss (Equation 1) is then computed from the four log-probabilities $\log \pi_\theta(y_w | x)$, $\log \pi_\theta(y_l | x)$, $\log \pi_{\text{ref}}(y_w | x)$, $\log \pi_{\text{ref}}(y_l | x)$ via the numerically-stable softplus formulation:

$$\mathcal{L} = \text{softplus}(-\beta \cdot [\log \pi_\theta(y_w) - \log \pi_\theta(y_l) - \log \pi_{\text{ref}}(y_w) + \log \pi_{\text{ref}}(y_l)]) \quad (2)$$

The reference model. DPO requires both the policy π_θ and the reference π_{ref} to be forward-passable on the same input. The reference is the SFT checkpoint, frozen. Naively this would require holding two copies of the 7B base model in memory. Instead, the implementation exploits the LoRA adapter architecture: the reference is exactly the SFT model with the LoRA adapters *disabled*, and the policy is the same model with the adapters *enabled*. The `enable_adapter_layers()` and `disable_adapter_layers()` methods on the `PeftModel` switch between these two states without instantiating a second model. The shared 4-bit base weights are loaded once and reused.

Hyperparameters. The DPO training uses $\beta = 0.1$ (gentle divergence allowed), one epoch, batch size 1, gradient accumulation 16 (effective batch size 16), learning rate 5×10^{-6} , and a cosine schedule with 5% warmup. The learning rate is intentionally an order of magnitude below the SFT learning rate: DPO updates should be small in scale, both because the policy starts near the SFT optimum and because aggressive updates risk catastrophic divergence.

A 5% held-out subset (518 pairs) is used for validation. Validation metrics after one epoch are: loss 0.435, accuracy 0.822 (the fraction of validation pairs where the policy correctly assigns higher reward to the chosen response), margin +0.92 (the mean difference in reward between chosen and rejected).

7.4 Evaluation

The DPO-trained checkpoint is evaluated on the same 3,112-record test set used for the SFT evaluation, with identical decoding parameters. Table 9 reports the aggregate metrics.

7.5 Comparison with the SFT Checkpoint

The aggregate improvements from DPO over the SFT checkpoint are real but small: mean character similarity rises from 0.859 to 0.864 (+0.5%), the high-similarity rate at threshold 0.8 rises from 0.731 to 0.745 (+1.4 percentage points), and token-overlap F1 rises from 0.817 to 0.823 (+0.6%). Exact match drops marginally from 0.225 to 0.220, which is consistent with the structure of DPO training: pushing the policy away from the reference necessarily trades some byte-perfect matches for broader similarity improvements when those trades come up.

Table 9: DPO patch-generator evaluation, pass@1 (greedy) on the 3,112-record test set.

Metric	Aggregate	CTSSB	CVEfixes
Exact match	0.220	0.220	0.224
Character similarity (mean)	0.864	0.871	0.756
High-similarity ≥ 0.8	0.745	0.758	0.529
High-similarity ≥ 0.5	0.972	0.981	0.828
Token-overlap F1 (mean)	0.823	0.826	0.773
Truncation rate	0.113	0.107	0.207

Aggregate numbers tell only part of the story. A more informative view comes from comparing the two models record-by-record. Table 10 reports the breakdown.

Table 10: Record-level comparison of SFT vs. DPO generations ($n = 3,112$).

Category	Count	Percentage
Identical generations	2,430	78.1%
DPO produced higher character similarity	418	13.4%
DPO produced lower character similarity	235	7.6%
Different text, same similarity	29	0.9%
Mean delta (DPO – SFT)		+0.0053
Mean delta when DPO improved		+0.086
Mean delta when DPO regressed		−0.084

The most striking number here is that **78.1% of generations are byte-identical between SFT and DPO**. This indicates that DPO at $\beta = 0.1$ modifies the model’s output for only a minority of records, leaving the rest unchanged. Within the modified subset (approximately 22% of records), 13.4% see character-similarity improvements and 7.6% see regressions, giving a ratio of approximately 1.78 improvements per regression. The mean magnitude of an improvement is +0.086 in character similarity; the mean magnitude of a regression is −0.084. The improvement count slightly outweighs the regression count, and the magnitudes are roughly symmetric, leading to the small but positive aggregate gain.

Qualitative impressions from the modified subset. The most interesting improvements are cases in which the SFT model made an outright incorrect API choice and the DPO model corrected it. For instance, on one CTSSB record the SFT model generated `requests.get(...)` while both the maintainer and the DPO model produced `requests.post(...)` — a verb-level semantic correction, not a formatting difference. On another record the SFT model produced a function call with seven incorrect `None` arguments, while DPO collapsed this to the correct two-argument form.

The largest regressions are mostly cases where the SFT model generated an output with similarity 1.000 (a byte-exact match to gold) and the DPO model drifted slightly to similarity 0.7 or so, producing a still-reasonable but no-longer-exact fix. One particular regression involves an early termination of generation under DPO that turned a multi-line block into a single-line truncated response. These regressions are honest limitations of synthetic-preference DPO without test-execution rewards.

7.6 Comparison with a Zero-Shot Baseline

The numbers reported so far compare two trained checkpoints (SFT and DPO) against each other. To put those numbers in context, this subsection compares both trained checkpoints

against the off-the-shelf base model that the trained checkpoints were built from: Qwen2.5-Coder-7B-Instruct, used zero-shot with no fine-tuning.

Setup. The baseline run uses the same test set (3,129 records, of which 3,123 produced valid prompts after window extraction — a slightly different count than the SFT evaluation due to a small difference in window-fitting tolerance), the same prompt template (Section 6), the same greedy-decoding parameters, and the same metric suite. The only thing that changes is the model: instead of loading the trained LoRA adapter on top of Qwen2.5-Coder-7B-base, the run loads Qwen2.5-Coder-7B-Instruct directly with no adapter attached. This isolates the effect of fine-tuning from any other factor.

Table 11 reports the metrics for the three configurations side-by-side.

Table 11: Comparison of the zero-shot baseline (Qwen2.5-Coder-7B-Instruct, no fine-tuning) against the SFT and DPO checkpoints, all evaluated on the same test set with the same prompt and decoding configuration.

Metric	Baseline	SFT	DPO
Exact match	0.066	0.225	0.220
Normalized EM	0.067	0.226	0.220
Character similarity (mean)	0.593	0.859	0.864
median	0.641	0.898	—
75th percentile	0.869	0.987	—
High-similarity ≥ 0.8	0.331	0.731	0.745
High-similarity ≥ 0.5	0.620	0.970	0.972
Token-overlap F1 (mean)	0.620	0.817	0.823
Truncation rate (heuristic)	0.686	0.111	0.113

The headline gaps are large. Character similarity rises from 0.593 (baseline) to 0.859 (SFT), an absolute improvement of 0.266 and a relative improvement of approximately 45%. Exact match more than triples, from 0.066 to 0.225. The high-similarity rate at threshold 0.8 more than doubles, from 0.331 to 0.731. Token-overlap F_1 rises from 0.620 to 0.817. Across every metric, the trained model is substantially closer to the maintainer-written reference patches than the zero-shot model is.

An important caveat about the baseline truncation rate. The baseline truncation rate of 0.686 indicates that more than two-thirds of the zero-shot generations hit the 256-token output budget without producing a clean stopping point. The reason is straightforward: the zero-shot model has never seen the localizer-window output convention, so it does not learn to stop at the closing code fence (“”). Instead, it tends to continue generating — often re-emitting the surrounding context, adding explanatory prose, or producing additional follow-up code — until the token budget is exhausted.

This inflates the apparent quality gap. A generation that gets cut off mid-statement scores low on character similarity regardless of whether the early tokens of the generation were useful. Some unknown fraction of the baseline’s similarity deficit is therefore attributable to format-adherence, not to fix quality. A fair reading is that the SFT and DPO training accomplish two related things at once: they teach the model to produce maintainer-style fixes for the bug at hand, *and* they teach the model to confine its output to the bracketed region. Both are real and useful learned behaviors, but they are different kinds of value and should not be confounded.

A separate experiment that constrained the baseline’s output more aggressively (for example, by stopping generation at the first “” token regardless of where it occurs, or by using a chat-formatted prompt that the instruct model is more comfortable with) would tighten this comparison. Such an experiment was not performed in the present work and is noted as a refinement for future evaluation.

What the comparison does and does not show. What it does show is that fine-tuning with the localizer-window recipe produces a clear and large improvement over the zero-shot baseline on every metric reported. What it does not show is precisely how much of that improvement comes from learning fix patterns specifically, as opposed to learning the output format. Both are real contributions of the training; separating them requires the format-controlled experiment described above.

8 Observations from Iterative Development

A number of observations arose during the iterative development of the system that are worth recording, because each shaped a specific design decision in the final system and is likely to be encountered by others building similar pipelines. These are reported as observations, not as contributions; the design choices they motivated are standard once recognized.

8.1 Observation 1: Token-Level F1 Is Not the Right Localization Metric

Symptom. An early version of the localizer reported token-level F_1 scores in the high 0.9 range, with no corresponding qualitative improvement when sampling outputs by hand.

Diagnosis. In a 7-line snippet with a single buggy line, approximately 85–90% of tokens belong to the not-buggy class. The token-level F_1 is dominated by the majority-class accuracy and gives high scores even to degenerate classifiers that predict not-buggy for every token. The metric does not measure the property of interest, which is whether the system surfaces the buggy line as a top prediction.

Resolution. Replace token-level F_1 with the line-level metrics described in Section 5.4: Top- K accuracy for $K \in \{1, 3, 5\}$ and Mean Reciprocal Rank. These correctly reward the system for ranking the buggy line near the top.

8.2 Observation 2: Augmentation Corrupted Localization Labels

Symptom. Localizer Top-1 accuracy plateaued well below expectations in an earlier training run that used data augmentation.

Diagnosis. Augmentations that modify the buggy-code text (whitespace perturbations, blank-line insertions, comment insertions, identifier renamings) shift line positions in the text but do not update the corresponding `changed_lines` label list. A blank-line insertion above line k moves the original line k to line $k + 1$ in the augmented text, but the label still refers to the original line k . The localizer thus receives a positive label at a position that is now actually a blank line, and a not-buggy label at the position that actually contains the bug.

Resolution. Track each training example’s `is_augmented` flag, and restrict the localization-loss computation to non-augmented samples only. Augmented samples continue to contribute to the classification loss (which does not depend on line positions) and to general representation learning in the shared backbone, but do not corrupt the localization head’s gradient. This is the form used in the final localizer.

8.3 Observation 3: The Classification Task Is Structurally Underspecified

Symptom. The auxiliary snippet-level classifier head collapses to predicting “buggy” for virtually every input, with F_1 of 0.665 and accuracy of 0.500 on a balanced held-out set.

Diagnosis. The classifier sees (`buggy_code`, `fixed_code`) pairs that differ by one or two tokens out of fifty to two hundred. The same code phrasing appears as a positive example in one record (the buggy version) and as a negative example in many other records (where it happens to be the fixed version, or unchanged context). There is no within-snippet signal that distinguishes which side of the fix the model is being shown.

Resolution. The task is genuinely ill-posed as formulated on this data; no amount of additional training would fix it. The proper fix is to train a separate classifier on data with non-microedit negatives — for example, random functions from CodeSearchNet that have no known association with a bug-fix commit. This is outside the scope of the present work and is discussed under future work in Section 9.1.

8.4 Observation 4: Exact Match Does Not Reflect Repair Quality on This Dataset

Symptom. An early evaluation of the SFT generator reported exact-match rates of around 0.225 and at first glance suggested the model was performing poorly. Qualitative inspection of the generations told a different story: most outputs were closely related to the gold fixes, often modulo small lexical differences or alternative-but-valid constructions.

Diagnosis. Exact match is a strict, binary metric. It gives no credit to a generation that differs from gold by a single space, by a renamed local variable, or by a semantically equivalent alternative function call. On a dataset dominated by small edits (89.2% of records are 5–9 lines), small lexical differences are the norm rather than the exception.

Resolution. Complement exact match with metrics that capture partial credit: character-level similarity (the `difflib.SequenceMatcher` ratio), token-overlap F_1 , and the high-similarity rate at multiple thresholds. The full metric suite tells a much more complete story than EM alone, as shown in the SFT evaluation (Section 6).

8.5 Observation 5: Two-Epoch Overfitting on the SFT Stage

Symptom. Training loss continued to decrease in epoch 2 of SFT training (0.381 to 0.211), but validation loss *rose* (0.367 to 0.402).

Diagnosis. The Qwen2.5-Coder-7B base model already speaks fluent Python from its pre-training; the role of SFT here is primarily to teach the localizer-window output format rather than to teach Python semantics. The output format is simple enough to be absorbed in a single epoch. Continued training causes the LoRA adapters to overfit to training-record idioms rather than to generalize.

Resolution. Track best validation loss across epochs, save the best-validation-loss checkpoint to a separate `best/` directory, and use that checkpoint for all subsequent evaluation and downstream training (DPO). In this case the `best/` checkpoint comes from epoch 1.

8.6 Observation 6: DPO Negatives Without Filtering Are Often Too Strong

Symptom. An initial pass through Stage 1 of the DPO sampling produced candidate negatives whose mean character similarity to gold was 0.736 (and 0.753 median on CTSSB specifically). This is well above the 0.50 range that would constitute genuinely “bad” fixes.

Diagnosis. CTSSB single-statement fixes have few realistic alternatives. Even with high-temperature sampling, the model mostly produces near-correct completions because there are not many distinct ways to fix a swapped operator or a renamed variable. Including these near-gold candidates as DPO negatives would teach the model to reject valid alternative fixes, which is the opposite of what is wanted.

Resolution. Apply a similarity threshold filter to retain only preference pairs in which the worst candidate has character similarity to gold below 0.70. The filter drops 59.1% of pairs but retains 10,371 pairs whose negative signal is genuinely informative. The mean character similarity of the retained negatives is 0.529, comfortably below the threshold.

9 Limitations and Future Work

This section discusses the principal limitations of the present system and outlines directions in which future work could address them.

9.1 Standalone Detector

As discussed in Section 5.6, the auxiliary classifier head built into the localizer does not function as a usable bug detector. A proper detector would be a separate model trained on a different data composition: function- or window-level Python code labeled by whether it is from a bug-fix dataset (positive) or from a corpus of code with no known association with a bug-fix commit (negative). The natural source for the latter is CodeSearchNet or a similar large random-code corpus.

Training such a detector is feasible in principle but introduces two complications that were not addressed in the present work. First, code in the present dataset (especially TSSB records) consists of short diff-context windows whose indentation and structural shape differs systematically from the standalone functions in CodeSearchNet. A naive classifier could exploit this structural difference rather than learning anything about bug presence. Controlling for the confound would require careful length- and indentation-matched sampling of the negative class.

Second, the detector serves only as a triage step in the pipeline and is not on the critical path for the present work, which operates in conditional mode (given a candidate buggy region, localize and patch). A properly designed detector would extend the deployment envelope but does not change the localization or generation quality once detection has occurred.

A complete detector with appropriate distributional controls remains future work.

9.2 No Execution-Based Validation

The DPO stage in the present work uses synthetically constructed preferences: gold fix versus model-sampled worst candidate. Aggregate improvements over the SFT baseline are real but modest (+0.5% character similarity, +1.4 percentage points in high-similarity rate). The ceiling of this approach is bounded by the quality of the preference signal.

A natural next step would be to replace synthetic preferences with preferences derived from test execution: for each generated patch, splice it into the original code, run the project’s test suite, and treat patches that pass tests as preferred over patches that fail. This is the recipe used in execution-based RL approaches to repair such as SWE-RL.

Implementing this for a Python repair system requires a sandboxed execution harness that can install dependencies and run arbitrary test code safely across many repositories. The infrastructure is engineering-heavy — per-project virtual environments, dependency installation timeouts, test discovery across heterogeneous project layouts, isolation against network-connecting or filesystem-modifying tests — and was outside the scope of the present work. It is the natural follow-up if higher repair quality is the goal.

9.3 The Generator Can Produce Patches That Introduce New Vulnerabilities

The CVE-2022-23071 example documented in Section 6.5 is the most important limitation to acknowledge in any deployment context. The generator produced an output that disabled TLS verification (`verify=False`) in place of the maintainer’s URL allowlist, addressing the superficial form of the original bug while introducing a new security weakness of comparable severity.

This is not a defect that can be patched by adjusting any single component. The model has no general capability to reason about security implications of edits, and similarity-based metrics do not penalize security-degrading alternatives strongly enough to prevent them from being

produced. Any deployment of an LLM-based repair system therefore requires human review of patches, particularly for security-sensitive code paths. The metric suite and qualitative examples reported in this work are intended to surface this risk rather than obscure it behind aggregate numbers, but they do not eliminate it.

9.4 Dataset Skew Toward Short Fixes

TSSB-derived records dominate the dataset and consist of short diff windows; 89.2% of records are 5–9 lines in length, and single-line edits constitute the overwhelming majority of training signal. The model is correspondingly strong on single-statement bugs but under-tested on multi-line, multi-statement repairs. CVEfixes provides longer records but in much smaller quantity (2,067 of 31,709 records, or 6.5% of the dataset).

A future iteration could rebalance the length distribution by sourcing additional medium-length records — for example by adapting Defects4J-style annotated bugs to Python, or by mining multi-line edits from larger commit-fix corpora.

9.5 Inputs Larger Than the Training Distribution

The present pipeline is designed for code windows similar in size to the training records: roughly 5–50 lines. When given a substantially longer input — for example a full Python module of 500 lines — behavior is undefined and the system will likely produce poor predictions, because the localizer has not been trained on inputs of that length and its tokenizer limit of 512 tokens may truncate non-trivially.

For deployment on whole files, an external chunking step (for example, AST-based function-level segmentation) is needed. Implementing such a step is straightforward but was not part of the present work.

9.6 Comparison with Prior Repair Systems

The character-similarity metric used in this report is a similarity-based metric. The standard metrics in published repair literature are based on test execution (plausibility, correctness), which require an execution harness as discussed above. Direct numerical comparison with prior repair systems is therefore not made in this report. The internal comparisons that *are* reported — the SFT-versus-DPO ablation in Section 7 and the zero-shot baseline comparison in Section 7.6 — are valid because they share the same test set, prompt template, and metric definitions. The present work is positioned as an investigation of effective fine-tuning recipes for Python bug repair rather than as a benchmark-driven study.

10 Conclusion

This report has presented CodeSentinel-LLM, an end-to-end pipeline for Python bug localization and patch generation built from two trained components: a GraphCodeBERT-based line-level localizer with LoRA adapters, and a Qwen2.5-Coder-7B-based patch generator trained in two stages with QLoRA followed by DPO. The combined dataset is a unified construction over TSSB-3M and CVEfixes v1.0.8, totaling 31,709 deduplicated Python bug-fix pairs split stratified by source.

The localizer achieves a line-level Top-1 accuracy of 0.806, Top-3 of 0.971, and Mean Reciprocal Rank of 0.888 on the held-out test set. The supervised generator achieves a mean character similarity of 0.859 between its outputs and maintainer-written reference patches, with 73.1% of generations exhibiting high similarity (above 0.8). The DPO-refined generator improves these to 0.864 and 74.5% respectively, with record-level inspection showing that improvements outnumber regressions by roughly 1.78 to 1. A zero-shot comparison against the same model family

without fine-tuning attains character similarity 0.593 and exact match 0.066 on the same test set, indicating that the fine-tuning recipe drives the substantial majority of the quality improvement over the off-the-shelf baseline.

Beyond the headline numbers, the project produced a set of observations about how to train and evaluate code-repair systems on small-edit data. Token-level F_1 was found to be misleading for localization and was replaced by line-level Top- K and Mean Reciprocal Rank. Exact match was found to systematically undersell quality on small-edit fixes and was complemented by similarity-based metrics. An auxiliary classification task was found to be structurally underspecified by the dataset and was disregarded in favor of conditional-mode operation. These observations are reported in detail because they shaped the final design and are likely to be encountered by others working with similar data.

The most important limitation acknowledged in this work is the possibility that the generator produces patches that are syntactically plausible but introduce new vulnerabilities, as documented in the CVE-2022-23071 example. Deployment of any LLM-based repair system therefore requires human-in-the-loop review, particularly for security-sensitive code. The metric suite and qualitative examples are intended to surface this risk while still providing useful aggregate signal about the system’s typical behavior.

References

- [1] Z. Feng et al., “CodeBERT: A Pre-Trained Model for Programming and Natural Languages,” in *Findings of EMNLP*, 2020.
- [2] D. Guo et al., “GraphCodeBERT: Pre-training Code Representations with Data Flow,” in *ICLR*, 2021.
- [3] B. Rozière et al., “Code Llama: Open Foundation Models for Code,” *arXiv:2308.12950*, 2023.
- [4] D. Guo et al., “DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence,” *arXiv:2401.14196*, 2024.
- [5] B. Hui et al., “Qwen2.5-Coder Technical Report,” *arXiv:2409.12186*, 2024.
- [6] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [7] T. Detrmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” in *NeurIPS*, 2023.
- [8] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, “Direct Preference Optimization: Your Language Model is Secretly a Reward Model,” in *NeurIPS*, 2023.
- [9] A. Silva, S. Fang, and M. Monperrus, “RepairLLaMA: Efficient Representations and Fine-Tuned Adapters for Program Repair,” *IEEE Transactions on Software Engineering*, 2025 (arXiv:2312.15698).
- [10] C. S. Xia, Y. Wei, and L. Zhang, “Automated Program Repair in the Era of Large Pre-trained Language Models,” in *ICSE*, 2023.
- [11] C. S. Xia and L. Zhang, “Automated Program Repair via Conversation: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT,” in *ISSTA*, 2024.
- [12] C. S. Xia and L. Zhang, “Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-Shot Learning,” in *ESEC/FSE*, 2022.

- [13] Y. Wei et al., “SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution,” *arXiv preprint*, 2025.
- [14] C. Richter and H. Wehr, “TSSB-3M: Mining Single Statement Bugs at Massive Scale,” in *MSR*, 2022.
- [15] G. Bhandari, A. Naseer, and L. Moonen, “CVEfixes: Automated Collection of Vulnerabilities and Their Fixes from Open-Source Software,” in *PROMISE*, 2021.
- [16] R. Widyasari et al., “BugsInPy: A Database of Existing Bugs in Python Programs to Enable Controlled Testing and Debugging Studies,” in *ESEC/FSE*, 2020.
- [17] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, “CodeSearchNet Challenge: Evaluating the State of Semantic Code Search,” *arXiv:1909.09436*, 2019.

A Hyperparameter Summary

Table 12 consolidates the principal hyperparameters used across the three training stages.

Table 12: Hyperparameters for all three trained components.

Parameter	Localizer	SFT Generator	DPO Generator
Base model	GraphCodeBERT	Qwen2.5-Coder-7B	Qwen2.5-Coder-7B
Base params	125M	4.39B	4.39B
Quantization	fp16	4-bit NF4	4-bit NF4
Adapter type	LoRA	QLoRA	QLoRA
LoRA rank	16	16	16
LoRA α	32	32	32
LoRA dropout	0.05	0.05	0.05
Trainable params	0.42M (0.33%)	40.4M (0.92%)	40.4M (0.92%)
Learning rate	2×10^{-4}	2×10^{-4}	5×10^{-6}
Schedule	cosine, 3% warmup	cosine, 3% warmup	cosine, 5% warmup
Batch size	16	4	1
Gradient accumulation	1	4	16
Effective batch	16	16	16
Max seq len	512	1024	1024
Epochs	6 (early-stop)	2 (best=1)	1
β (DPO)	—	—	0.1

B Prompt Template

The full prompt template used during both supervised and DPO training, and at inference time:

```

### Task:
Fix the bug in the following Python code. The buggy region is marked
with # <BUG_START> and # <BUG_END>. Output ONLY the corrected version
of that marked region -- do not output the surrounding context.

### Buggy code:
'''python
{window_buggy_code_with_sentinels}
'''

```

```
### Fixed region:  
'''python
```

The model is trained to continue from this prompt with the fixed region followed by a closing fence (``). End-of-sequence conditioning at the end of the target makes generation terminate naturally at the fence.